MEDIA-INFORMANT

Installation Instructions

Hosting the web application on a Synology NAS with DSM Web Station, Nginx, and PHP 8.4

Document type	Operator installation guide	**Platform**	Synology DSM + Web Station
Runtime	PHP 8.4 (Nginx backend)	**Database**	SQLite 3 via sqlite3 + pdo_sqlite
Scope	NAS-side hosting prerequisites	**Date**	6 November 2026

This guide records the Synology-specific work required to host Media-Informant. Install the packages, create the application root and catalog shares, grant the `http` group the rights it needs, then configure a PHP 8.4 profile, a Web Service, and an alias portal under Web Station. Apache is not used; the HTTP backend is Nginx.

Why the PHP module path still matters

DSM ships PHP 8.4 with an `extension_dir` that points at `/usr/local/lib/php84/modules`. That directory is empty. The real `sqlite3.so` and `pdo_sqlite.so` live inside the PHP 8.4 package under `/volume1/@appstore/PHP8.4/usr/local/lib/php84/modules/`. Set that path in the Web Station profile (Section 3.1). The CLI php.ini edit in Section 5 is only a fallback if the GUI value did not stick.

1. Installing Components

Complete these items in Package Center and Control Panel before touching PHP configuration.

Item	Notes
SSH	Enable in Control Panel → Terminal & SNMP. You will edit files as a user with sudo if the optional Section 5 fallback is needed.
Web Station	Required. Serves the application and owns the PHP-FPM profiles. Open it later from Package Center.
PHP 8.4 package	Install from Package Center. Requires DSM 7.2 or later. Do not confuse this with a Docker PHP image.
HTTP server backend	**Nginx only.** Apache HTTP Server is not used for Media-Informant. Select Nginx when the Web Service is created in Section 3.2.
Shared Folder / RAID volume	Application source and media files must exist on at least one RAID volume. On a typical single-volume NAS that path is `/volume1`.
Data Folders	One or more shared folders that host the media file content used as catalog sources (for example `/volume1/Video` and `/volume1/Music`). They must be shared folders so you can grant the **`http`** group Read/Write access. A plain directory that is not a shared folder cannot receive that group ACL.
Application Root	A folder that must be created inside the top-level `web` directory (itself created by Web Station). The folder is named `media-informant` and holds the application source code. Absolute path: `/volume1/web/media-informant`.

Throughout this guide, **`http`** is the DSM system group that Web Station PHP-FPM runs as. It is not the HTTP protocol. Do not confuse it with the web server name.

2. Folders and user / group permissions

Open **Control Panel** and go to the **Shared Folder** section.

Step 2.1 — web folder, read only for the http group

Select the `web` folder that is auto-generated by Web Station. Choose **Edit**, then the **Permissions** tab. Filter the list on **local groups** and grant **Read only** to the `http` group. Save.

Step 2.2 — catalog data sources, Read/Write for the http group

Find the first shared folder that contains the media data that will be a catalog source in the app (for example `Video`). Again choose **Edit**, then the **Permissions** tab. Filter on **local groups** and give the `http` group **Read/Write**. Save. Repeat for every additional catalog data source (Music, a USB share, and so on).

> **Why catalog shares need Read/Write**
>
> Read only on a catalog share is enough to list files. PHP Tools (rename and other writes) need Read/Write on the parent folder of each media file. Assign that right to the `http` group on every catalog share you plan to write. Apply to this folder, sub-folders and files if the dialog offers it. Do not chown the whole library to the web user if Plex or other packages also use those shares.

Optional confirmation from SSH:

```
sudo synoacltool -get /volume1/Video
```

With Tools enabled you want a line equivalent to `group:http:allow:rwxpdDaARWc--:fd--`.

3. Configure Web Station

Open **Web Station from inside Package Center** (Package Center → Web Station → Open). Do not rely on a leftover Main Menu shortcut from an older profile.

3.1 Create the Script Language Settings profile

Create a new **Script Language Settings** item. It uses **PHP 8.4** and is named **Media Apps**.

Settings tab

Enable **PHP cache**. Give PHP `open_basedir` a custom value. The minimum is the application root; append every catalog source folder. Multiple folders are delimited by a colon. Example with a Video catalog source:

```
/volume1/web/media-informant:/volume1/Video
```

Add further sources the same way, for example `/volume1/web/media-informant:/volume1/Video:/volume1/Music`. Keep any system paths the UI already listed, then append the application root and the catalog folders.

Extensions tab

Enable both SQLite modules. These are the modules that made the already-installed application able to use SQLite.

Module	Required state	Purpose
`sqlite3`	Enabled	Native SQLite3 API used by the application.
`pdo_sqlite`	Enabled	PDO driver. Enable even if the app currently uses the native API.

Core tab

Scroll until you find the `extension_dir` variable. Change its value to the corrected absolute path of the real PHP 8.4 modules directory:

```
/volume1/@appstore/PHP8.4/usr/local/lib/php84/modules
```

Do not leave it on `/usr/local/lib/php84/modules` — that directory is empty. Do not point it at `libsqlite3.so`. PHP loads `sqlite3.so` and `pdo_sqlite.so`; those modules already link against the system library. The ini setting `sqlite3.extension_dir` is only for SQLite plugins loaded later via `SQLite3::loadExtension()`.

Save all changes to create the new Script Language Settings profile.

3.2 Create the Web Service

Create a new Web Service entry:

Field	Value
Name	media-informant
PHP profile	Media Apps (the profile from 3.1)
Document root	web/media-informant
HTTP backend	Nginx (not Apache)

Apply the changes.

3.3 Create the alias portal

Select the **Web Portal** tab and press **Create**. Select the **media-informant** service. Portal type **must** be **Alias portal**. Set the alias to `media-informant`. That alias is the tail end of the URL users will use to open the app in a browser, for example `http:///media-informant`. Apply the changes to create the portal.

4. Place the application files

Open **File Station**. Browse into the `web` folder. From the computer, use a **drag-and-drop** of the local `media-informant` folder onto that `web` NAS folder so the source files and folders copy in one operation. This is the only way to upload the entire `media-informant` folder at one time. Do not upload the contents file-by-file into a mismatched path.

```
/volume1/web/media-informant/
```

The virtual-host document root and this directory must match. The **`http`** group must be able to read the PHP files and write the SQLite database path the app uses under this folder. At this stage the app may still fail on SQLite until the Media Apps profile from Section 3.1 is saved and attached.

5. Edit PHP Configuration

This step is optional. It is only required if the `extension_dir` value set in the Core tab of the Media Apps profile (Section 3.1) did not take effect. Open the CLI `php.ini` and check that variable. If it already shows the @appstore path, stop. If it still points at the empty modules directory, set it by hand.

Step 5.1 — Confirm where the modules actually are

SSH into the NAS and run:

```
ls -ld /usr/local/lib/php84 /usr/local/lib/php84/modules
ls -l /volume1/@appstore/PHP8.4/usr/local/lib/php84/modules/sqlite3.so \
      /volume1/@appstore/PHP8.4/usr/local/lib/php84/modules/pdo_sqlite.so
php84 --ini
```

Expected result: `/usr/local/lib/php84/modules` is empty. The two `.so` files exist under `/volume1/@appstore/PHP8.4/usr/local/lib/php84/modules/`. Note the **Loaded Configuration File** printed by `php84 --ini`. On this installation it is:

```
/usr/local/etc/php84/cli/php.ini
```

Step 5.2 — Set extension_dir only if the GUI did not

Open that file as root. Comment the existing `extension_dir` line with a leading semicolon and add a line that points at the @appstore modules directory. Quotation marks around the path are optional.

```
;extension_dir = /usr/local/lib/php84/modules
extension_dir = /volume1/@appstore/PHP8.4/usr/local/lib/php84/modules
```

In the same file, or in `/usr/local/etc/php84/cli/conf.d/extension.ini` if you prefer a drop-in, add:

```
extension = sqlite3.so
extension = pdo_sqlite.so
```

> **Do not reference libsqlite3.so in php.ini**
>
> PHP loads `sqlite3.so` and `pdo_sqlite.so`. Those modules already link against the system `libsqlite3.so`. The ini setting `sqlite3.extension_dir` is only for SQLite plugin modules loaded later via `SQLite3::loadExtension()`.

Step 5.3 — Verify CLI

```
php84 -m | grep -i sqlite
```

Success is both names listed, with no “Unable to load dynamic library” warning:

```
pdo_sqlite
sqlite3
```

CLI PHP rereads the ini on every invocation. No service restart is required for command-line checks. CLI and Web Station do not share this file; the site uses the Media Apps profile from Section 3.1.

6. Confirm the application

Open the alias URL in a browser (the portal from Section 3.3). The app should create or open its SQLite database without a missing-driver or undefined-class error, and it should be able to list files under the catalog shares granted in Section 2.

Optional PHP check from a page in the site document root:

```
<?php
echo 'sqlite3: ' . (extension_loaded('sqlite3') ? 'yes' : 'no') . PHP_EOL;
echo 'pdo_sqlite: ' . (extension_loaded('pdo_sqlite') ? 'yes' : 'no') . PHP_EOL;
print_r(PDO::getAvailableDrivers());
```

Run that file through the website, not through `php84` on SSH, if you want to test the same PHP-FPM profile the app uses. Remove the file when you are done.

7. What will overwrite this work

These edits are on package-owned files. They are the correct place to change, but they are not permanent across updates.

Change	Lost when
Media Apps extension_dir and extension checkboxes	Usually kept, but re-check after a PHP 8.4 or Web Station update. Repair can reset the Core value.
CLI extension_dir and extension = lines	PHP 8.4 package update or Repair in Package Center. Only needed if Section 5 was used.
Files under /volume1/@appstore/PHP8.4/	Any reinstall or upgrade of the PHP 8.4 package. Do not patch those .so files; point at them.
System libsqlite3.so under /lib or /usr/lib	DSM update. Do not edit those libraries.
http group ACL on Video / Music / web	Survives package updates. Recheck if a shared folder is recreated or permissions are reset.

After a PHP 8.4 update, reopen the Media Apps profile (Settings, Extensions, and Core) and re-run Section 5.3 only if the Core `extension_dir` was wiped.

8. Operator checklist

1	Web Station, PHP 8.4, and Nginx available. Apache is not used.
2	Application and media live on a RAID volume, typically /volume1.
3	Application root exists at /volume1/web/media-informant.
4	Catalog data folders are shared folders (Video, Music, and any others).
5	On Shared Folder → web → Permissions, local groups: http group is Read only.
6	On each catalog share → Permissions, local groups: http group is Read/Write.
7	Web Station opened from Package Center.
8	Script Language Settings profile Media Apps uses PHP 8.4, PHP cache on.
9	open_basedir includes /volume1/web/media-informant and every catalog folder, colon-separated.
10	Extensions: sqlite3 and pdo_sqlite enabled.
11	Core extension_dir is /volume1/@appstore/PHP8.4/usr/local/lib/php84/modules.
12	Web Service media-informant uses Media Apps, document root web/media-informant, Nginx.
13	Alias portal created for the media-informant service; alias is media-informant.
14	Source uploaded in one drag-and-drop of the media-informant folder into File Station → web.
15	Section 5 php.ini edit only if the GUI extension_dir did not stick.
16	Browser test of the alias URL opens the app and can read catalog shares.

End of guide. Components and paths are Section 1. Folder ACLs are Section 2. Web Station (Media Apps, Nginx service, alias portal) is Section 3. Upload is Section 4. The php.ini edit is only a fallback in Section 5.